实验二：软件设计与建模

项目：酒店预订系统

姓名：商世雄 学号：241820122 日期：2026-11-06

1 项目说明

本实验选题为酒店预订系统，沿用实验一的需求，完成系统结构、类、数据存储和界面的设计。报告中的界面为原型，业务后端将在后续实验中实现。

实验一的requirements.md共有26条需求，其中22条已确认，包括18条功能需求、3条非功能需求和1条约束。其余4条涉及超时、停售修改和性能测试条件，本次在4.4节补充具体做法，不改动实验一的原始记录。

系统分为顾客端、酒店工作台和平台管理台。游客可以查酒店，顾客登录后可以预订和管理自己的订单。工作人员维护本酒店的房型、价格和库存，处理预订及入住退房。管理员审核酒店、管理工作人员账号、处理异常订单，并查看全平台的统计和日志。各类账号只能操作权限范围内的数据。

主要业务流程是：顾客查询酒店和空房，选择房型并提交预订，酒店确认后办理入住，住宿结束后办理退房。系统还要处理修改、取消、拒绝、超时、未入住和酒店临时停业等情况，并保留相应记录。

住宿费用在线下结算，系统不接入支付、退款、评价、优惠券、地图、会员积分或第三方渠道。房量只按房型管理，不分配具体房间，也不处理清洁维修、提前入住、延迟退房和续住。通知保存为平台待办，由工作人员线下联系。

2 软件结构与模块划分

系统分为展示层、应用层、领域层和持久化层，业务模块放在同一个应用中，使用同一个数据库。预订时要同时生成订单、占用多个日期的库存，只要其中一步失败就要全部撤销。因此，本项目没有采用微服务，而是把这些操作放在同一个数据库事务中处理。

模块	负责的功能	对应需求
M01 账号与权限	顾客注册、各类用户登录、账号启停；检查操作者是否有权访问目标订单或酒店	REQ-001、023
M02 酒店与房型	酒店审核启停、房型资料、每日房量、价格和停售设置	REQ-002、003
M03 查询与报价	按城市、日期、人数等条件查房，计算逐夜价格和总价；提交时重新检查	REQ-004、007
M04 预订处理	创建订单、确认拒绝、修改取消、入住退房；处理重复请求和旧页面操作	REQ-005—016、020

模块	负责的功能	对应需求
M05 库存与价格	预留和归还各晚房量，计算改单后的库存变化，保存成交价	REQ-003、006、007、011
M06 异常处理	检查超时、未入住和停业影响，调用预订服务完成相应处理	REQ-010、015、017
M07 统计与日志	汇总有效预订和间夜数，按权限查询订单历史和操作日志	REQ-018、019
M08 数据保存与备份	保存业务数据，维护关联和唯一约束，执行备份、恢复及通知重试	REQ-020、024

页面只收集输入和展示结果，具体业务交给应用服务。应用服务先检查权限和订单状态，再调用库存服务、Repository等完成处理。这样，顾客页面、酒店页面和定时任务不会各写一套扣房规则。统计模块只读取数据，不修改订单和库存。

UML部分共9张图，涉及7种类型。组件图、两张类图和部署图说明系统结构；用例图、状态图和活动图说明业务行为；两张时序图分别说明创建预订和修改预订时的调用顺序。各图均用PlantUML绘制，保留源文件和导出图片。

3 类与数据设计

订单由Booking表示，各晚的成交价和占用情况保存在BookingNight中。RoomDay则记录某房型某一天的供给、占用和价格。例如一笔订单连住三晚，就对应三条BookingNight记录，并影响三个日期的RoomDay。

BookingService负责组织一次订单操作，InventoryService负责计算房量变化。AuthorizationPolicy检查身份和数据归属，TransitionPolicy检查订单状态和操作时限。

三类用户共用用户表，通过role区分，工作人员另绑定一个hotelId。订单中的房型必须属于该订单的酒店。逐夜成交价与酒店当前房价分开保存，酒店调价时只更新后续报价，不改已经生成的订单。

数据表	主要内容和限制
app_user	id为主键，账号不能重复；保存密码哈希、角色、酒店编号和启用状态。工作人员只能绑定一个酒店
hotel	保存名称、城市、地址、联系方式、审核及营业状态；time_zone使用IANA时区名，如Asia/Shanghai；按城市和状态建立索引
room_type	id为主键，hotel_id关联酒店并建立索引；capacity（可住人数）和total_rooms（房间总数）均为正整数
room_day	以房型编号和住宿日期为联合主键；保存供给、占用、价格、停售标记和版本。已占用量不能为负或超过供给，供给不能超过房型总房数

数据表	主要内容和限制
booking	保存顾客、酒店、房型、入住离店日期、间数、人数、联系人、状态、总价和版本；另记确认计时起点及截止时刻。离店日期须晚于入住日期，间数和人数均为正整数
booking_night	以订单编号和住宿日期为联合主键，关联对应的房型日历，保存成交单价、间数和占用状态；同一订单同一晚只有一条记录
request_record	操作者、操作类型和请求键的组合不能重复；保存请求内容哈希及处理结果，供重复提交时返回原结果
audit_log	保存操作者、对象编号、酒店、动作、前后状态、结果、原因和时间，提供只读查询
notification_task	保存关联订单、通知内容、处理状态和重试次数，通知失败后可以重试

订单查询主要按顾客、酒店、状态和入住日期进行，因此计划建立(customer_id,created_at)、(hotel_id,status,check_in)索引。超时任务使用(status,expires_at)索引。日志按酒店或操作者与时间组合建立索引。查询结果分页返回，实际效果在后续测试中检查。

取消订单、停业或停用账号都不删除历史订单。订单修改后保留新的逐夜明细，旧日期、价格及联系人变化记入日志。日志中的电话先脱敏，不保存密码和整份请求。日志保留180天，由系统定期清理，管理页面不提供修改和删除功能。

4 主要业务处理

4.1 跨日库存与价格

库存按住宿夜计算，入住日计入，离店日不计入。例如11月10日入住、12日离店，只占10日和11日的房量；预订两间，则这两晚各占两间。当天还能出售多少间，等于当天供给减去仍应占用的房量。

预订提交成功后，订单进入待确认，同时预留全部住宿夜的房量。酒店确认和办理入住时不再扣一次库存。如果任一晚无房、停售、没有日历数据，或处理中写入失败，整笔预订都不成功，已做的修改全部撤销。查询和提交之间可能有人订走房间，所以提交时还要重新检查。

订单总价是各晚单价相加后再乘房间数。例如两晚分别为299元和359元，订一间共658元。提交时带上顾客已经接受的报价版本；若当前价格变了，先展示新价格，让顾客重新确认。生成订单后保存成交价，后续酒店调价不影响这笔订单。

4.2 并发预订和重复提交

创建预订时，先登记请求键，检查酒店营业状态和房型，再锁定所需日期的库存，核对价格和余量。检查通过后，写入预留、订单、逐夜价格、处理结果、操作日志和通知待办。这些写入由同一个UnitOfWork管理，要么一起提交，要么全部撤销。失败原因在事务回滚后另行记录，避免日志也跟着被撤销。

身份校验和查询已有结果可以先做，但登记新请求键必须放在事务中，并由数据库唯一约束防止重复。如果两个相同请求同时到达，后一个等待前一个完成，再读取结果；前一个回滚时则允许重试。网络超时不能直接当作预订失败，顾客可以用原请求键查到结果，避免另建一笔订单。

两人同时订最后一间时，不能都根据页面上的旧余量下单。处理程序按房型编号、日期的固定顺序锁住库存记录，读取最新余量，确认变更后不会超过供给才更新。

下单时对酒店和房型使用共享锁，酒店停业和房型维护使用与其冲突的锁，避免检查完营业状态后酒店刚好停业。改单先锁订单，再依次锁酒店、房型和日历。停业操作只更新酒店状态，之后逐单取消时分别开事务，不在持有酒店锁的同时反向锁住全部订单。

请求键与操作者、操作类型一起判断是否重复。同键、同内容返回原结果；同键但内容不同，返回409和REQUEST_KEY_CONFLICT，原结果不变。顾客确实要再订一笔时，使用新的请求键。“无房”这类已处理完的结果可以保存，临时服务器故障则允许原键重试。

修改、确认和取消都带上页面读取到的订单版本expectedVersion。实际处理时版本或状态已变，就返回409并要求刷新。权限也由服务端重新检查，不能只靠页面隐藏按钮。如果酒店先确认成功，顾客刷新后仍可在允许时限内取消；如果顾客先取消成功，酒店就不能再确认。

4.3 修改、取消和库存释放

“待确认”表示顾客已提交订单，酒店还没有确认。这时可以修改日期、房型、间数和联系人。修改时锁定新旧日期涉及的库存，逐日计算新占用量减去原占用量。重叠日期的房量没有变化，就保留原来的占用，不再扣减。

例如原订单在10日和11日各占一间，改成只住10日，就保留10日的占用，归还11日的一间。如果想延住到12日，但12日没有房，则整次修改失败，原订单、价格和预留都不变。保存前要让顾客接受新价格；保存成功后重新计算30分钟的酒店确认时限。

酒店已经确认的订单只能改联系人，不重新计时。日期、房型或间数有变，需要按规则取消后重订。改单限于原酒店，换酒店也需要重新预订。

顾客可以取消待确认订单。已确认但尚未入住的订单，只能在入住日酒店当地18:00前取消，18:00整及以后不再允许。取消或拒绝后归还尚未消费的预留，重复操作不再次归还。正常退房不恢复已经住过的房量。

到了酒店当地入住日次日00:00，已确认但仍未入住的订单标为“未入住”。一晚订单没有后续房量可归还；三晚订单只归还后两晚。BookingNight用HELD、CONSUMED、RELEASED区分预留、已消费和已归还，过去的住宿夜不重新出售。

4.4 停业、超时及补充规则

酒店临时停业后，先停止接受新预订。管理员逐笔处理待确认和已确认订单，填写原因后取消，并在同一事务中归还预留。通知先记为待办，失败后重试。已入住订单不直接改成已退房，由平台与酒店联系住客处理。恢复营业需管理员手动启用。

房型停售与酒店停业分开处理：单纯停售不取消原订单，但不能在停售日期增加占用。具体规定如下。

编号	处理事项	采用的规则
P01 / REQ-010	30分钟截止	到期即不能确认或修改，不因后台任务延迟而延长。到期前已合法确认的订单，不再进入超时处理
P02 / REQ-017	停售和停业时的修改	停售日期可保持或减少原有占用，不能增加；其他日期仍检查可售和余量。酒店停业后不允许改日期、房型和间数。联系人修改及取消按原状态规则处理
P03 / REQ-021、022	性能测试条件	使用第6节的固定环境，查询和预订分别做20并发、100次测试，失败及超时均计入统计

订单创建或修改成功时，将本次计时起点加30分钟，存入expiresAt。处理确认时，锁定订单后读取服务器时间，必须早于expiresAt，恰好到期也拒绝。后台每5秒检查一次到期订单；确认、修改等写接口也做这项检查，发现已到期就先转为已超时、归还一次库存，再返回超时提示。

每家酒店保存IANA时区名，例如Asia/Shanghai。入住和离店按当地日期记录，创建、截止和日志时间存为UTC时间戳，显示时再换算。取消的18:00和未入住判定的00:00都按酒店当地时间计算。

查询时不填日期，可以浏览酒店和房型资料，但不显示整段总价和是否可订，价格筛选也不可用。只填了入住或离店中的一个日期，就提示补齐。

统计按计划入住日计入，开始日和结束日都包含。例如查询11月1日至30日，1日和30日入住的有效订单都算在内，不把跨区间订单拆成每晚统计。只统计已确认、已入住和已退房订单。

易用性测试安排5名首次使用者，每人分别用电脑和手机完成一次查询及预订。两端分别计时，从填写查询条件到看到预订成功结果为止，出错和重试也算在内。测试人员不代操作，至少4人两端都在4分钟内完成，并记录设备、浏览器版本和用时。

5 界面设计

桌面页面左侧放导航，右侧显示当前功能。顾客主要用查房、填写预订和我的订单；酒店工作人员主要用房型日历、预订处理和入住退房；管理员主要用酒店审核、员工账号和异常处理。

查房和预订页面同时显示逐夜单价与整段总价，避免把每晚价格和住宿总价混在一起。库存页面把供给、已占用和剩余分列。订单状态除颜色外还写出文字，操作按钮则按状态安排。手机端单独做订单详情页，保留查看明细、修改和取消入口。

界面源文件为diagrams.net的.drawio格式，文字和控件可以分别编辑，共导出16张PNG。另外保留了可点击的HTML原型，演示页面跳转和操作反馈，不连接真实后端。

原型中可以查看日期填错、提交前改价、改单无房、重复取消、库存调低和酒店停业后的提示。实际的登录鉴权、数据库锁、请求去重和定时任务仍要在后续实验实现。

UI编号	界面 / 对应模块	功能说明
UI-01	账号登录与注册 / M01	顾客注册，三类身份登录；工作人员由管理员创建，密码不在原型中真实保存
UI-02	酒店组合查询 / M03	城市、日期、人数、房型和整段价格；日期错误、无结果给出可理解提示
UI-03	酒店房型详情 / M02、03	展示营业状态、逐夜单价和余量、全程总价；不按离店日占库存
UI-04	提交预订 / M04、05	联系人、人数间数、逐夜价；确认报价后提交，调价要求再次接受
UI-05	我的订单 / M04	仅本人订单、状态筛选、跳转修改和详情，不展示他人数据
UI-06	修改待确认订单 / M04、05	左右对照原/新方案；重叠夜只保留一次，失败仍保留原单
UI-07	订单详情与历史 / M04、07	版本与状态、逐夜快照、取消截止、只读操作历史、电话脱敏
UI-08	房型资料 / M02	容量和房间总数、所属酒店、跳转日历，不管理房间号
UI-09	逐日价量停售 / M02、05	总量、已占用、剩余、价格和销售状态分列；拒绝调减至已有占用以下
UI-10	确认与拒绝 / M04、06	拒绝原因、刷新状态、确认不重复扣房，到期不允许确认
UI-11	入住退房 / M04	查询本酒店订单、现场核验结果、重复操作拦截，不保存证件照片
UI-12	酒店审核启停与异常 / M02、06	停业影响预览、原因、逐单取消、通知待办、已入住人工协调
UI-13	工作人员账号 / M01	创建和停用，唯一酒店绑定，停用不删历史，禁止扩大权限
UI-14	统计 / M07	本酒店/平台角色范围；3单9间夜示例，按计划入住日纳入，包含查询区间首尾日期
UI-15	审计日志 / M07	操作者+时间+类型组合条件、只读记录、前后状态、结果原因及脱敏
UI-16	移动端订单详情 / M04	手机查看逐夜明细和修改取消入口；底部触达区域与桌面端相同规则

6 非功能设计

检查内容	设计做法	验证要求
权限和密码	服务端角色+归属校验、受限数据库账号、参数化SQL、加盐单向哈希、日志脱敏；Cookie会话采用HttpOnly/Secure/SameSite并校验CSRF	三角色至少30组非法请求全部拒绝且业务数据不变；直接请求不能绕过页面限制
查询性能	city/status、房型日历联合索引、分页、按区间聚合最小余量，不缓存可售库存作为最终提交依据	使用本节统一测试基线，20并发100次，至少95次在2秒内正确返回
预订性能与正确性	短事务、固定锁序、最后一间锁后重查、请求唯一约束；通知在提交后处理	同环境20并发100次，至少95次3秒内明确结果，全部不得超卖；同时包含充足库存成功场景
可靠性	原子事务、幂等重试、审计与通知待办、每日备份和隔离恢复演练	每24小时备份；2小时内恢复至最近成功备份，RPO≤24小时；正常事务不可部分提交
易用性	字段级错误提示、总价口径说明、状态相关操作、手机详情适配、清晰的破坏性操作确认	5人分别测试桌面和手机，至少4人两端均在4分钟内独立完成；计时包含出错和重试

后续性能测试按下表准备环境和数据。

项目	测试配置
服务端	8核逻辑CPU、16GB内存、100GB SSD，Ubuntu 22.04 LTS 64位；一个应用实例与一个数据库实例运行在同一测试主机，不部署其他高负载服务
软件	OpenJDK 17、Spring Boot 3.3、PostgreSQL 16、Nginx 1.24；正式测试时记录具体补丁版本和配置，不在测试中途更换
压测端与网络	独立的4核、8GB主机，JMeter 5.6.3；千兆局域网，往返延迟不超过10毫秒；客户端从发出请求至完整收到响应计时
数据规模	10000家酒店、每家5种房型，共50000房型；100000条订单；每种房型准备连续30个住宿日，共1500000条日历记录，库存与订单预留一致
查询请求	20名并发用户，每人5次，共100次；1、2、3、7晚各25次，条件覆盖城市、人数、房型、价格和无结果场景。至少95次在2秒内正确返回，错误和超时均计不达标
预订请求	先做4轮20并发的充足库存预订，共80次；再以20个不同请求键竞争同房型同区间最后一间，共100次。前80次应成功，最后20次仅1次成功、19次明确返回无房；至少95次3秒内返回业务结果，不得超卖
执行方式	查询与预订分开测试，每项开始前恢复同一初始数据。记录每次条件、耗时、结果和库存；不把失败剔出分母，不用全部拒单替代成功预订场景

本次已检查图形渲染、原型演示和文件完整性。表中的性能、安全、恢复和首次使用测试，要等业务系统实现后再做。

7 AI工具使用记录

本次通过OpenCode使用GPT-6.1 Sol（openai/gpt-6.1-sol）做AI辅助设计。输入材料是实验一需求和实验二任务要求，主要用来生成UML草稿、界面原型及检查脚本。

使用环节	生成内容	检查和修改
A01 模块和UML	列出8个模块，生成类、状态和交互等9张图的PlantUML代码	修正状态图语法，调整过宽的服务类图；检查订单、日历和价格记录的关系
A02 界面原型	生成16页界面、可编辑源文件和HTML演示	调整表格文字，检查各页面的跳转及错误提示，补充超时和停售修改的演示
A03 文件检查	生成渲染、格式和原型交互检查脚本	检查图能否生成、文件是否齐全、演示是否出现脚本错误；业务系统测试另做

初稿及后续修改分别提交到Git。初稿标签为lab2-ai-draft，第一轮技术检查的标签为lab2-technical-check。制作目录中的审查与版本记录保留了具体过程，业务设计的本人核对记录尚待补充。

业务部分还要重点核对改单时的重叠日期、酒店停业与下单同时发生的情况，以及各角色能操作哪些订单。

已完成的技术修改如下。

修改项	原来的问题	处理结果
R01 状态图语法	中文状态名未加引号，第一次渲染报错	加上引号并保留英文别名，9张图均可生成
R02 类图布局	服务依赖全排在一行，图宽2545像素，放入报告后文字太小	调整布局，将依赖分行，图宽降到约1400像素，调用关系不变
R03 界面表格	较长文字可能挤到下一列	按列宽调整字号；16页源文件能在diagrams.net打开，控件未超出画板
R04 原型操作	静态图片看不出出错后如何处理	先检查7组常见操作，再补充5组边界情况，共12组演示通过，未出现JavaScript异常
R05 图片和压缩包	部分图缩小后难读，制作文件不应混入提交包	调整图页方向和大小；打包35个文件，并与提交目录逐字节比对
R06 报告排版	标题和粗体过于突出，正文有重复说明	改用常规宋体和普通白底版式，缩小标题，删去装饰及重复语句
R07 边界规则	超时、停售修改、测试环境和日期范围的处理不够具体	补上严格截止、停售日期不增加占用等规则，同时修改图和界面说明

这次使用中，AI比较适合先列出类和模块，或者生成PlantUML代码，省去了从空白文件开始写的步骤。但结果仍要检查。例如服务类图能够生成，却因为太宽而不适合放进报告；状态图也出现过语法错误。

后续可以继续按“整理需求、生成草稿、检查和修改、保存版本”的顺序使用。语法、文件数量和部分原型操作可以自动检查，但权限、库存及状态规则仍需对照需求判断，不能完全交给脚本。

8 需求对应关系

需求	对应设计	图 / UI
REQ-001	M01，角色与对象归属	用例/服务类；UI-01、13
REQ-002	M02，酒店审核营业状态	组件/领域类；UI-03、12
REQ-003	M02、05，房型与日历供给	领域类；UI-08、09
REQ-004	M03，组合条件及全程库存	活动；UI-02、03
REQ-005	M04，原子建单	创建时序；UI-04
REQ-006	M05，排序锁定与净变化	领域类/两张时序；UI-06、09
REQ-007	M03、05，逐夜报价与快照	创建时序；UI-03、04
REQ-008	M04，RequestRecord唯一键	服务类/创建时序；UI-04、07
REQ-009	M04，确认拒绝	状态；UI-10
REQ-010	M06，严格截止及写接口补做超时处理	状态/服务类；UI-05、10
REQ-011	M04、05，改单净变化	修改时序；UI-06
REQ-012	M04，取消状态及截止	状态；UI-07
REQ-013	M04，现场核验入住	状态/用例；UI-11
REQ-014	M04，正常退房	状态；UI-11
REQ-015	M06，未入住只放未来夜	状态/服务类；UI-11
REQ-016	M04，策略与版本防护	状态/两张时序；UI-05—11
REQ-017	M06，停业逐单处置；停售日期禁止新增占用	组件/状态；UI-09、12
REQ-018	M07，按计划入住日统计	组件；UI-14
REQ-019	M07，审计与授权查询	领域类/部署；UI-07、15
REQ-020	M08，外键唯一与事务	领域类/服务类；UI-13
REQ-021	查询索引及固定测试基线	部署；UI-02
REQ-022	短事务、充分库存及最后一间竞争压测	创建时序/部署；UI-04
REQ-023	授权、哈希与脱敏	服务类/部署；UI-01、15
REQ-024	原子性、备份和恢复	两张时序/部署；报告第6节
REQ-025	首次使用任务和跨端适配	UI-02、04、16；报告第6节
REQ-026	范围约束	报告第1节；所有图/UI均不含支付退款

9 实验小结

这次设计中，库存不能只用一个“剩余房间数”概括。订单连住几晚，就要分别检查几天；改期时还要扣除订单自己原来的占用，取消时也要防止重复归还。将这些情况放到逐夜明细中，比只看订单总状态更容易说明清楚。

因此，订单处理集中在BookingService中，库存服务计算每一晚的变化，页面和定时任务都调用这些服务。后续实现时，先完成查询、预订、确认、取消和入住退房，再按本报告列出的条件检查并发预订、超时及停售修改。

后附UML和界面图，提交目录中保存原尺寸图片及PlantUML源文件。

图1 组件图

结构性图。三个客户端通过业务模块访问数据库，预订与库存使用同一事务。

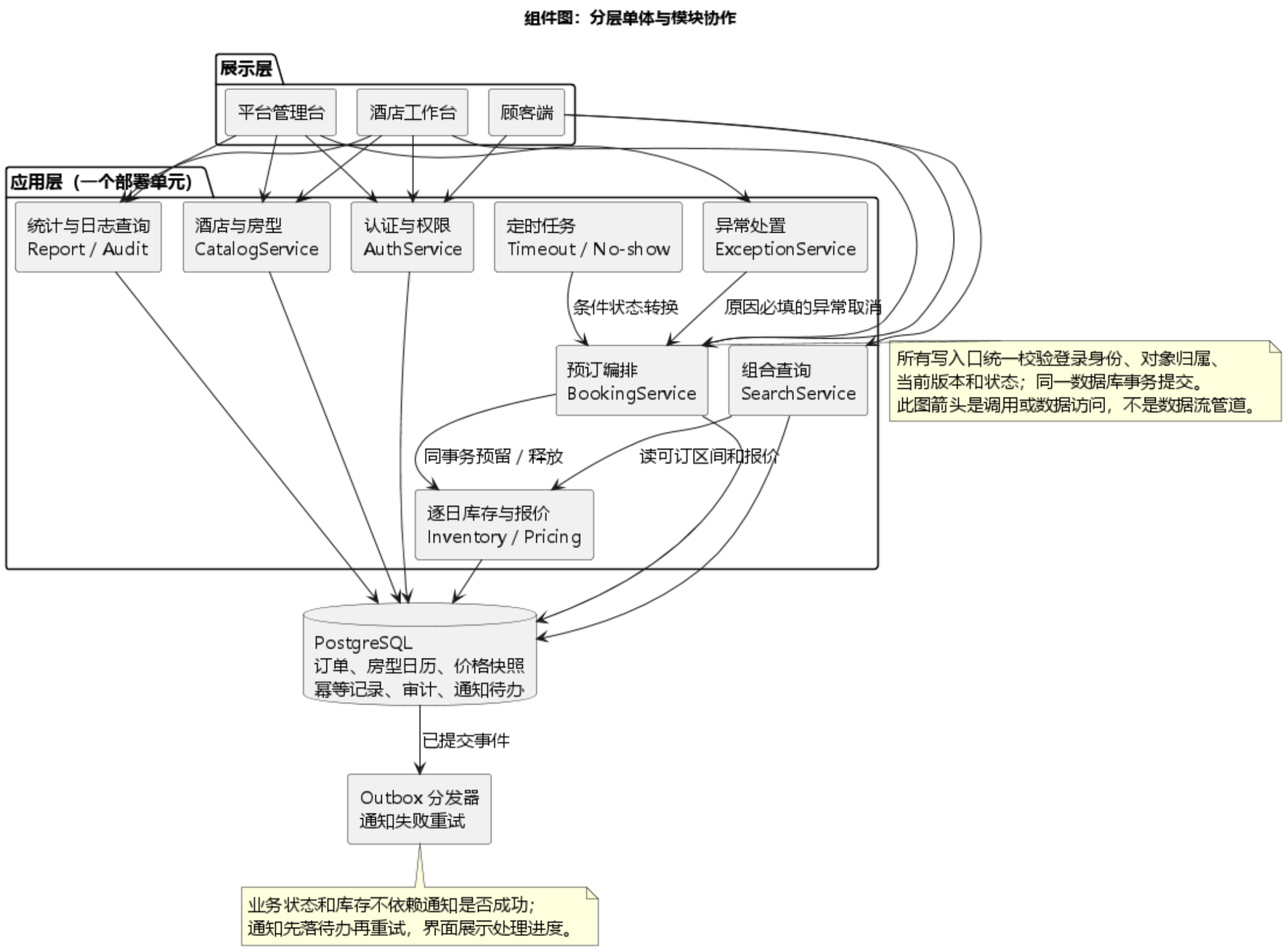


图2 领域类图

结构性图。一笔订单关联多个住宿夜，每晚对应一个房型日历记录。图中标出主要关联和唯一约束。

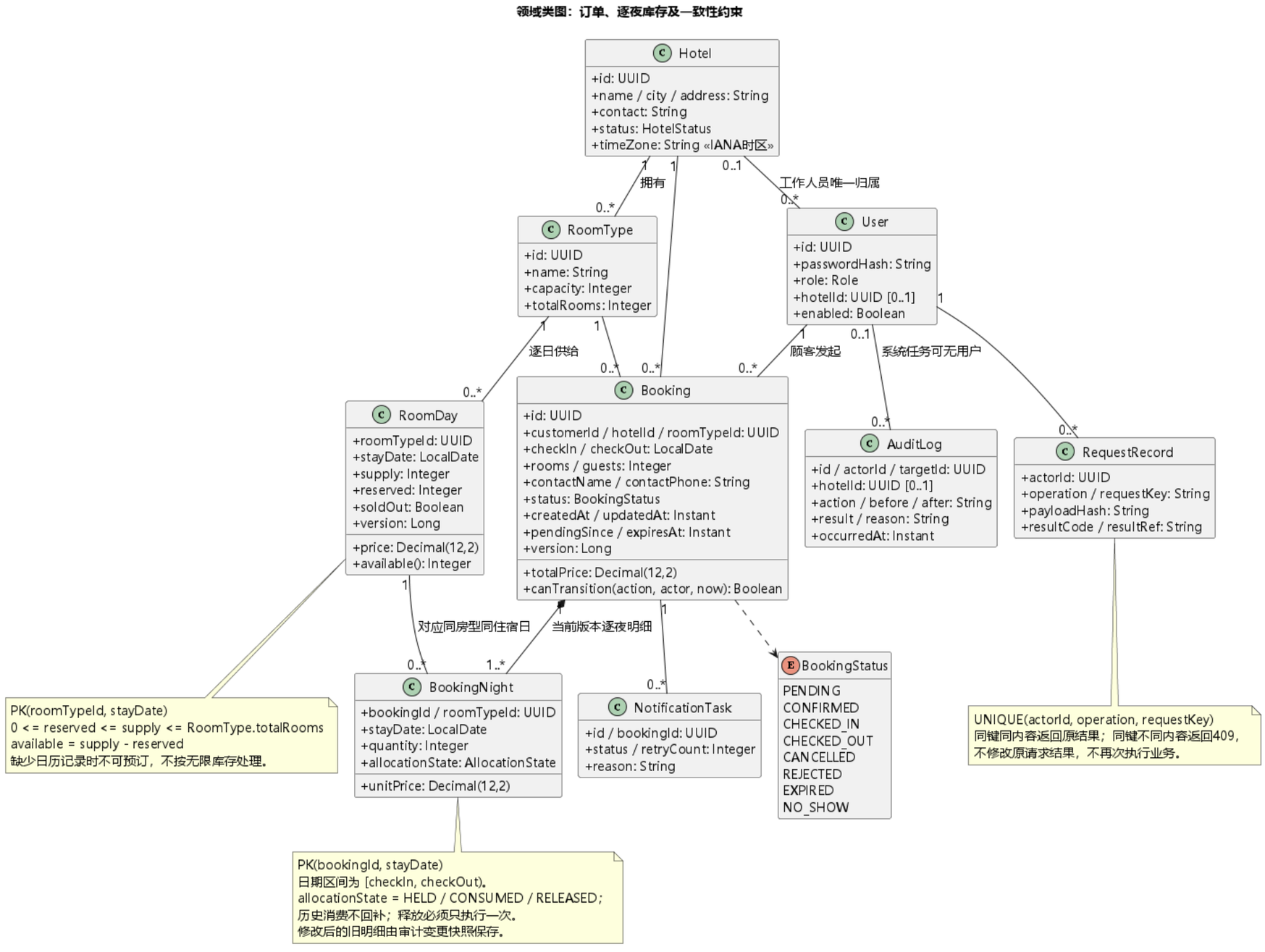


图3 应用服务类图

结构性图。BookingService调用权限、状态、库存和数据访问接口，定时任务也从该服务处理订单。

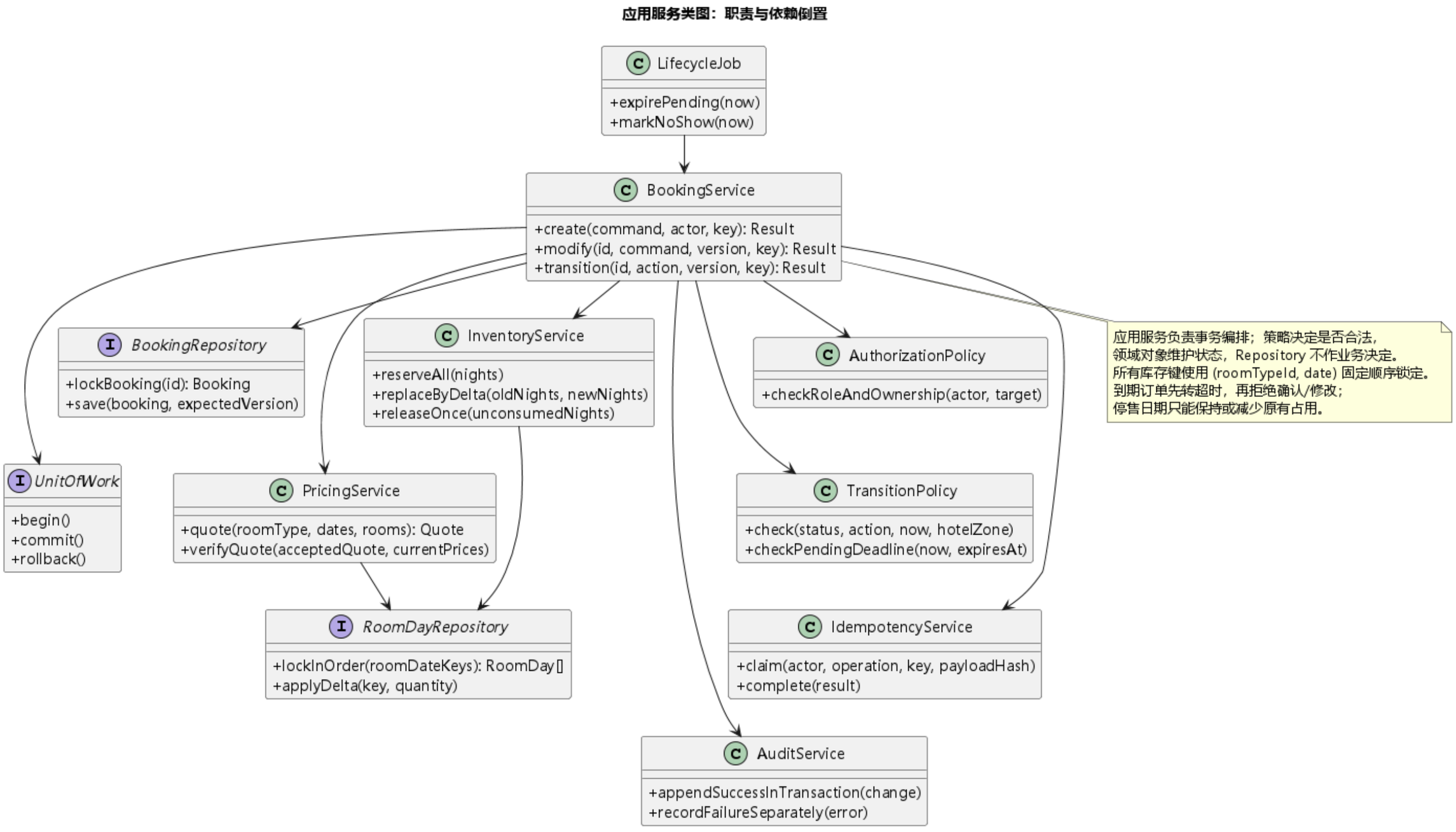


图4 用例图

行为性图。分别列出游客、顾客、酒店工作人员和平台管理员的操作范围。

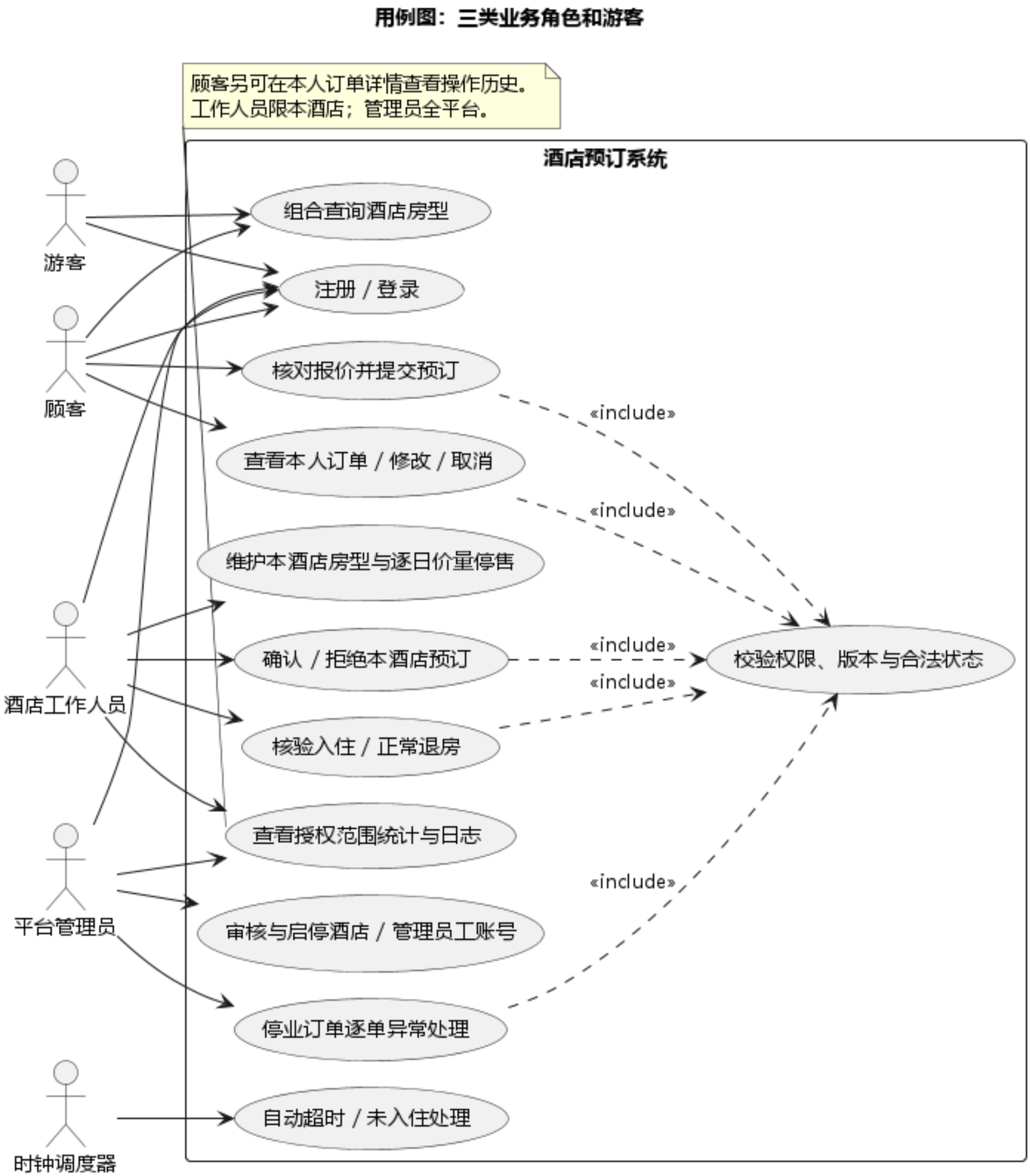


图5 订单状态图

行为性图。箭头旁注明状态变化的条件和房量处理，已取消等终态不能再办理入住。

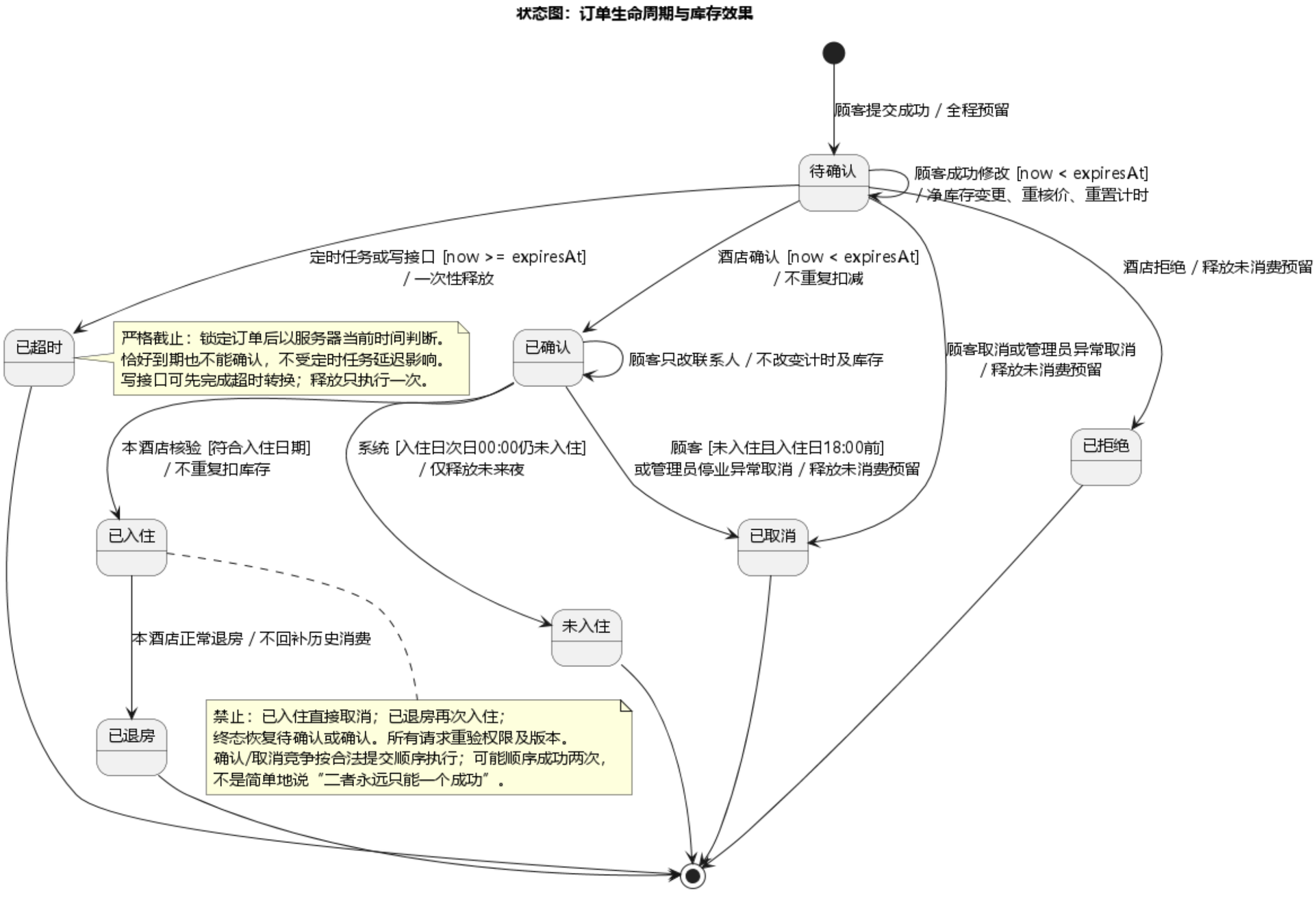


图6 预订活动图

行为性图。从输入查询条件开始，列出无房、改价、重复请求和提交失败时的处理。

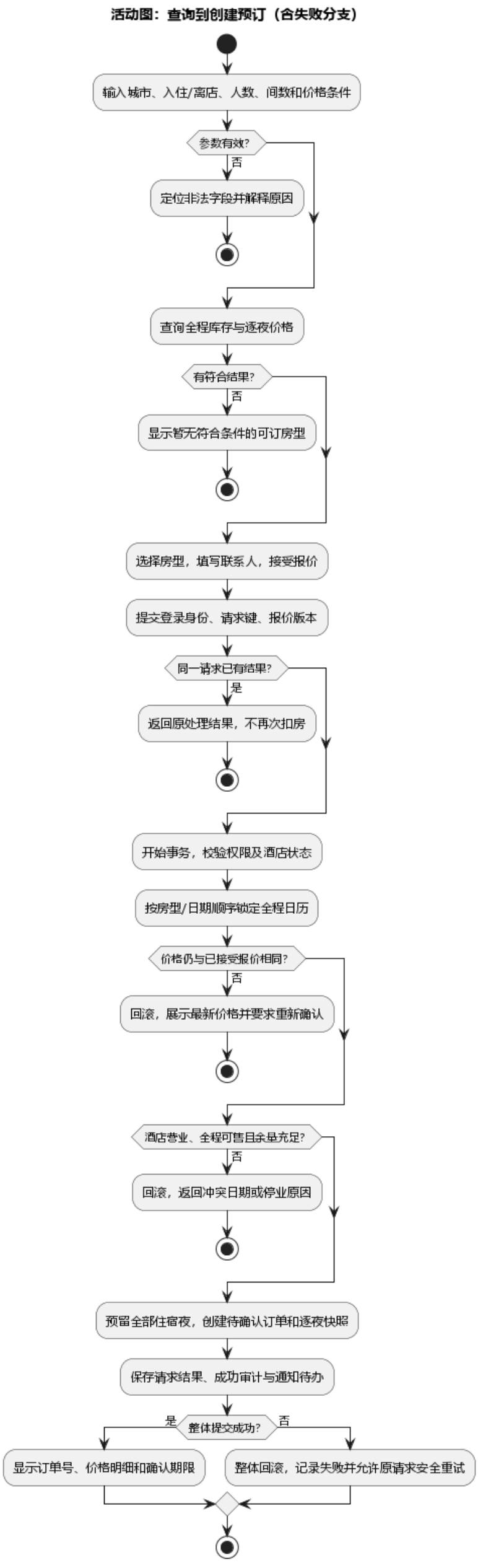


图7 创建预订时序图

交互性图。依次检查请求、价格和库存，再一次提交订单及全部住宿夜的预留。

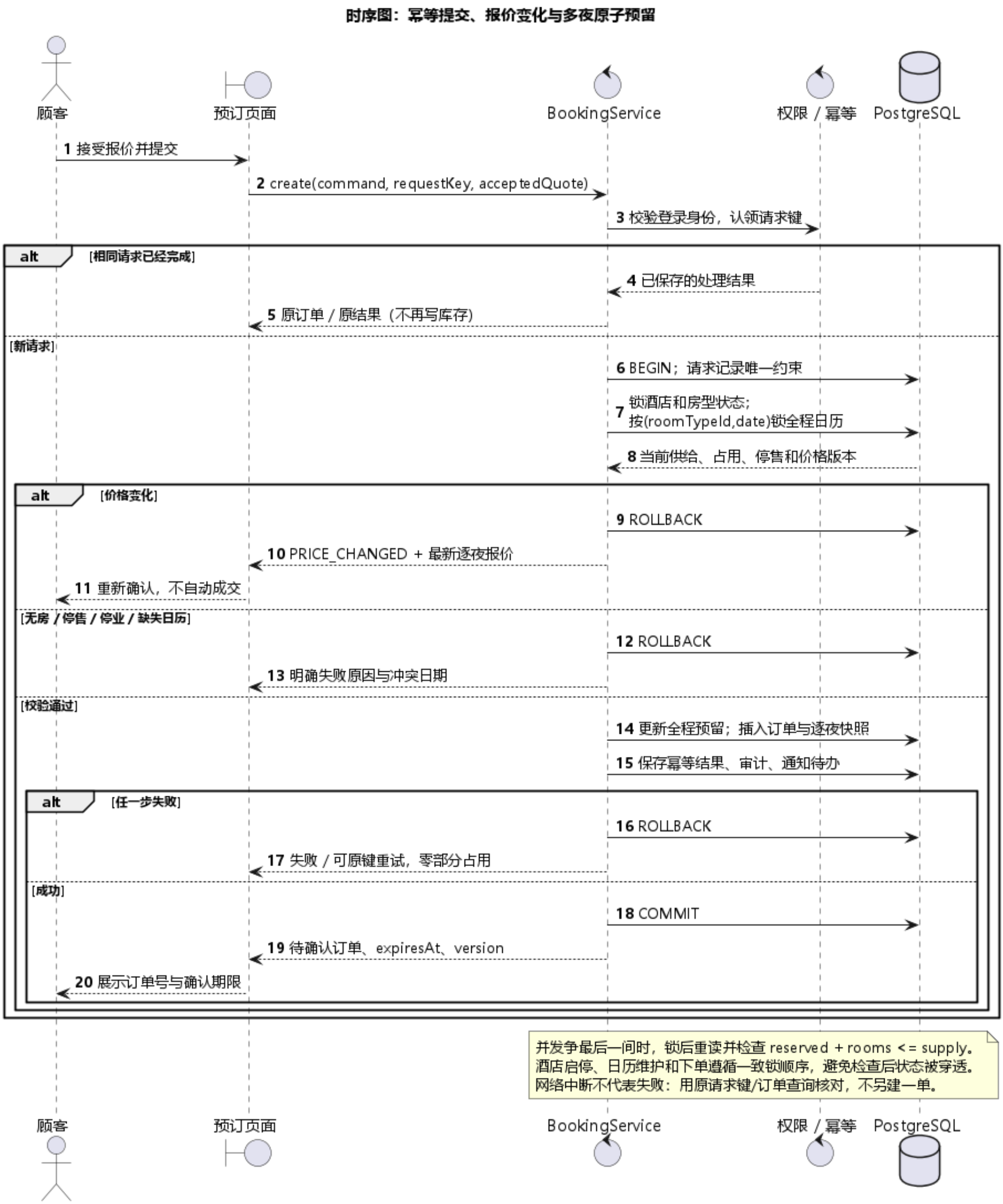


图8 修改预订时序图

交互性图。改单先比较新旧日期的占用；新日期无房时，原订单和预留保持不变。

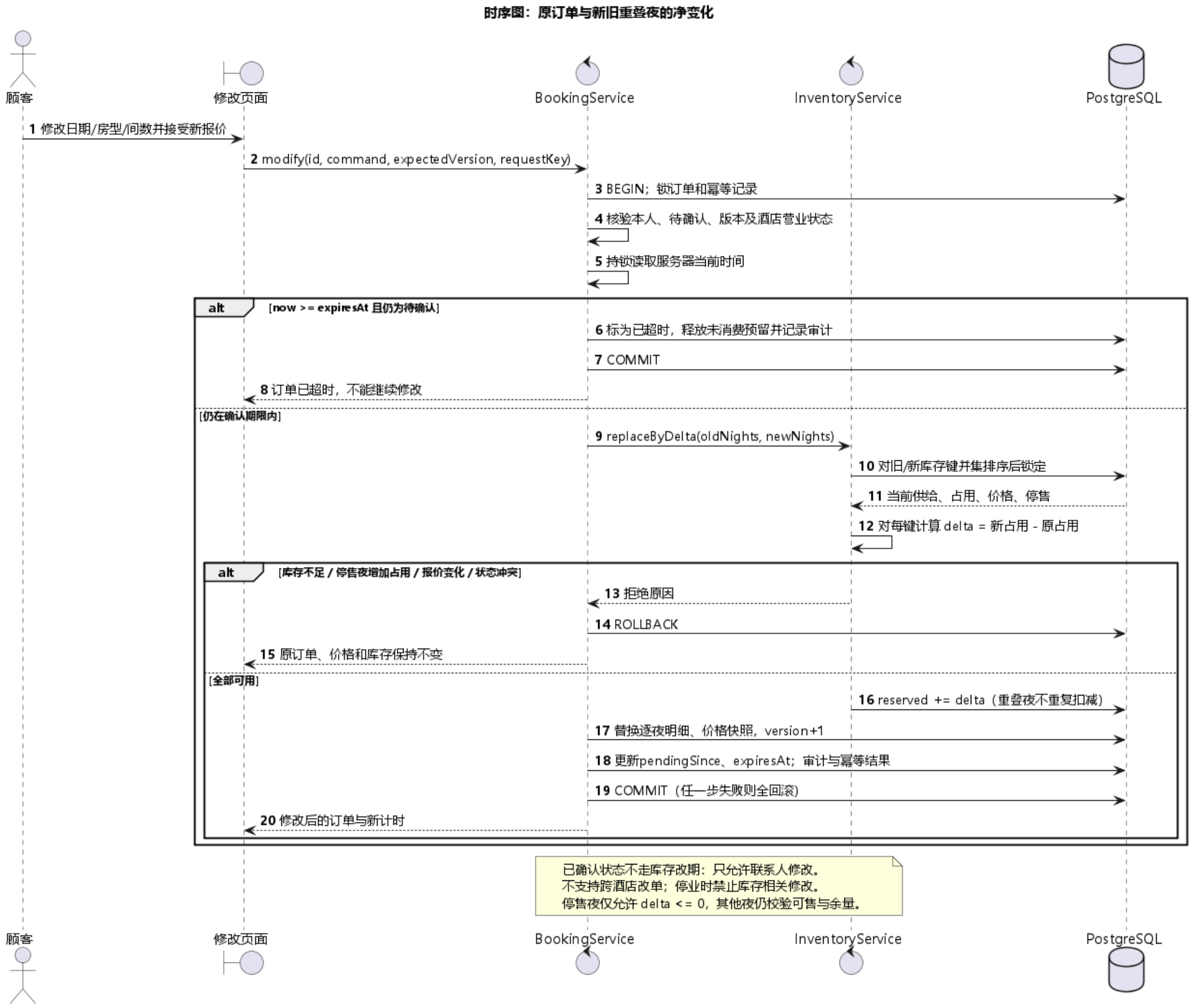
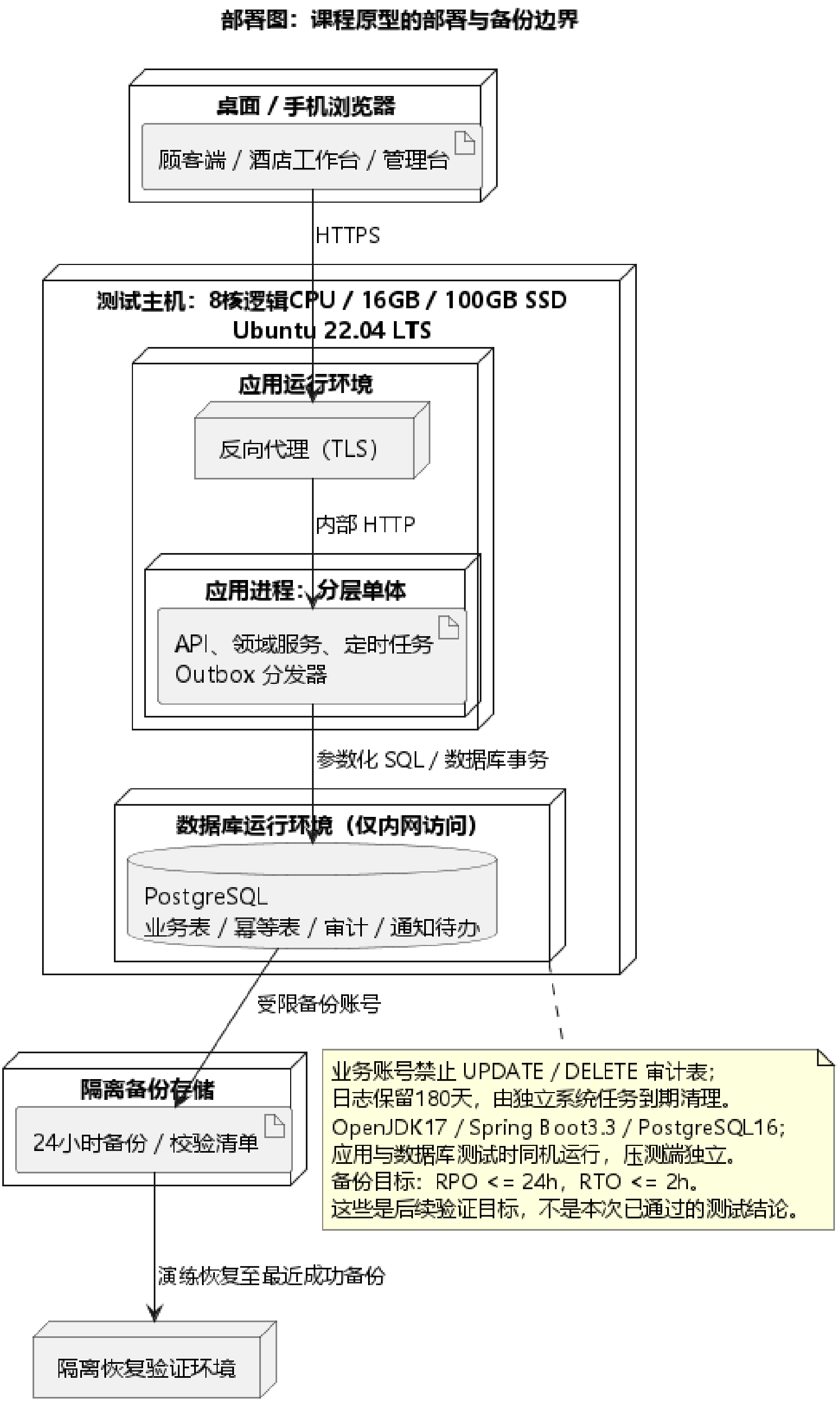


图9 部署图

结构性图。测试时应用与数据库同机运行，压测端独立，备份放在隔离存储中。



UI-01 账号登录与注册

顾客端，对应REQ-001 / 023。顾客自行注册；酒店工作人员账号由平台管理员创建。



UI-02 寻找下一段旅程的落脚处

顾客端，对应REQ-004 / 007。按城市、日期、人数、房型和整段住宿价格组合查询。



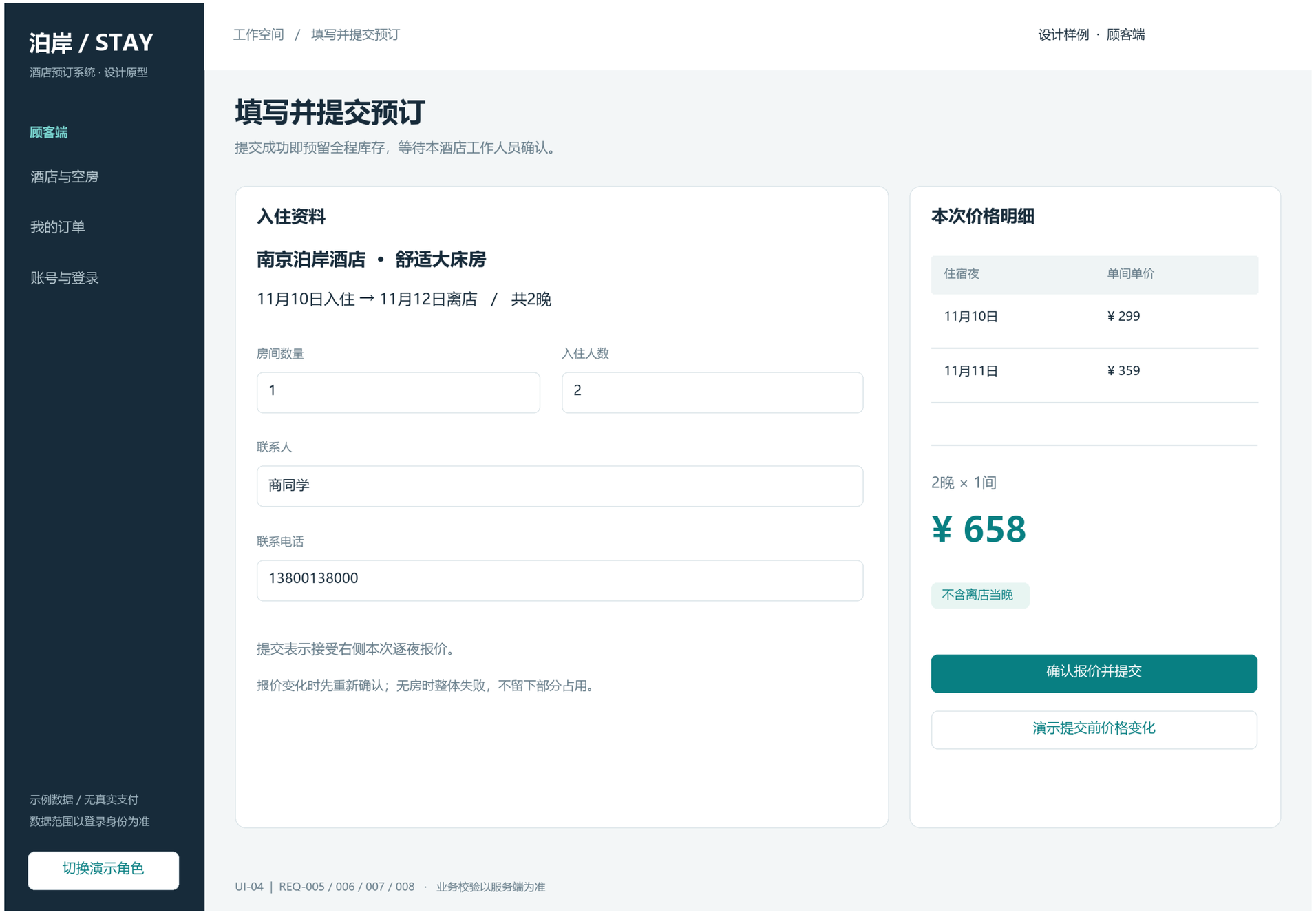
UI-03 南京泊岸酒店

顾客端，对应REQ-002 / 003 / 004。南京市鼓楼区湖南路18号 • 联系电话 025-8000-1234



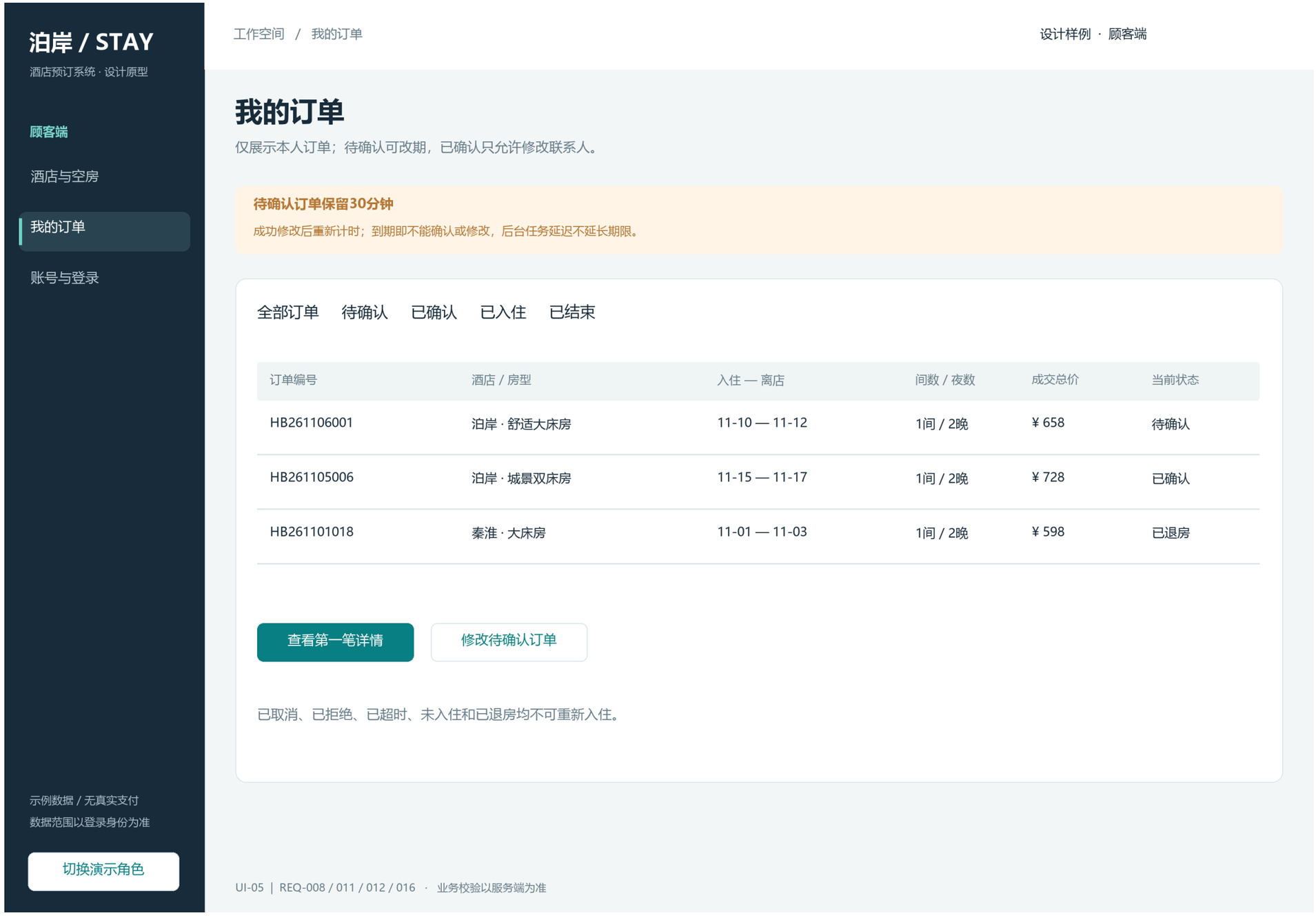
UI-04 填写并提交预订

顾客端，对应REQ-005 / 006 / 007 / 008。提交成功即预留全程库存，等待本酒店工作人员确认。



UI-05 我的订单

顾客端，对应REQ-008 / 011 / 012 / 016。仅展示本人订单；待确认可改期，已确认只允许修改联系人。



UI-06 修改待确认订单

顾客端，对应REQ-006 / 007 / 011。新日期无房时保留原订单；先核价，成功后一次性处理新旧预留。



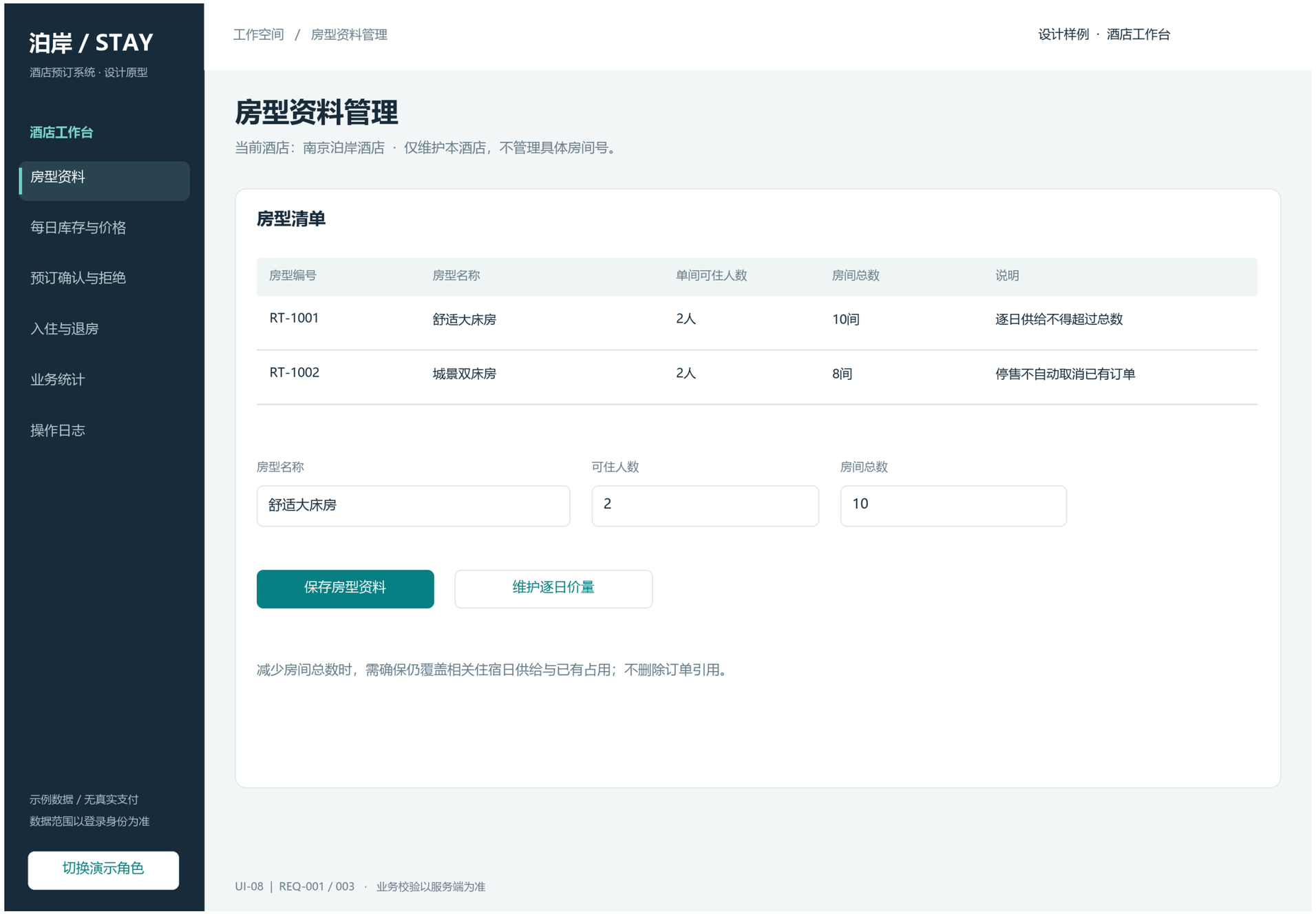
UI-07 订单详情与操作历史

顾客端，对应REQ-008 / 012 / 016 / 019。HB261106001 • 服务端版本 v3 • 当前状态和操作结果以最新数据为准



UI-08 房型资料管理

酒店工作台，对应REQ-001 / 003。当前酒店：南京泊岸酒店 • 仅维护本酒店，不管理具体房间号。



UI-09 每日库存、价格与停售

酒店工作台，对应REQ-003 / 006 / 007 / 017。房型 RT-1001 • 舒适大床房 | 分清供给总量、已占用与剩余量。

泊岸 / STAY

酒店预订系统 · 设计原型

酒店工作台

房型资料

每日库存与价格

预订确认与拒绝

入住与退房

业务统计

操作日志

示例数据 / 无真实支付

数据范围以登录身份为准

切换演示角色

工作空间 / 每日库存、价格与停售

设计样例 · 酒店工作台

每日库存、价格与停售

房型 RT-1001 · 舒适大床房 | 分清供给总量、已占用与剩余量。

剩余房量 = 当日供给总量 − 仍应占用的房量

停售日期不能增加占用，原订单可减少或释放占用；已生成订单的价格快照保持不变。

住宿日期	供给总量	已占用	剩余	单晚价格	销售状态
2026-11-10	3间	2间	1间	¥ 299	可售
2026-11-11	1间	0间	1间	¥ 359	可售
2026-11-12	3间	1间	2间	¥ 329	停售

11月10日供给总量

3

11月10日单晚价格

299

销售状态

可售

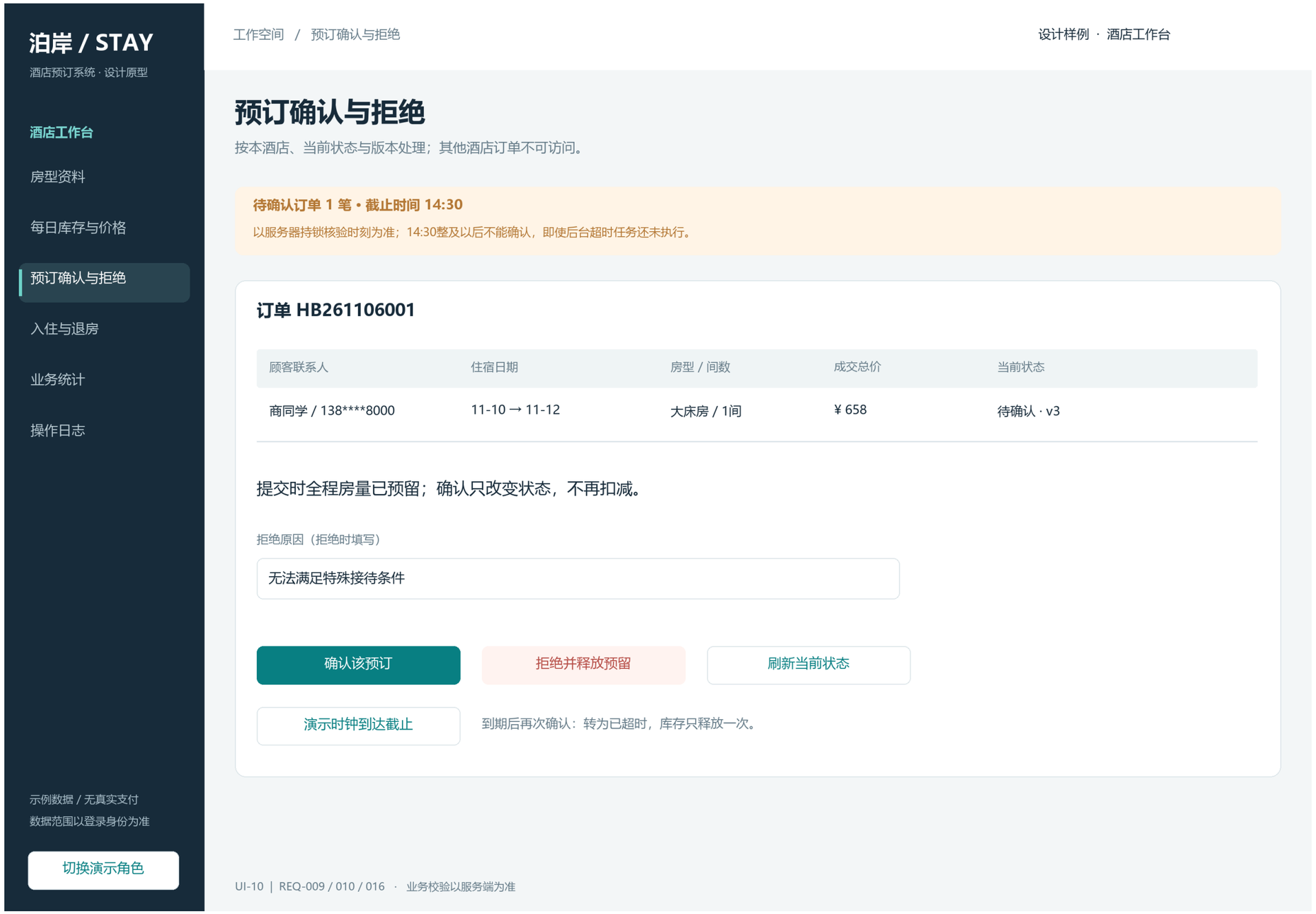
保存当日变更

示例：总量3、占用2，还能再售1间；若把总量调成1，必须拒绝，不能负库存。

UI-09 | REQ-003 / 006 / 007 / 017 · 业务校验以服务端为准

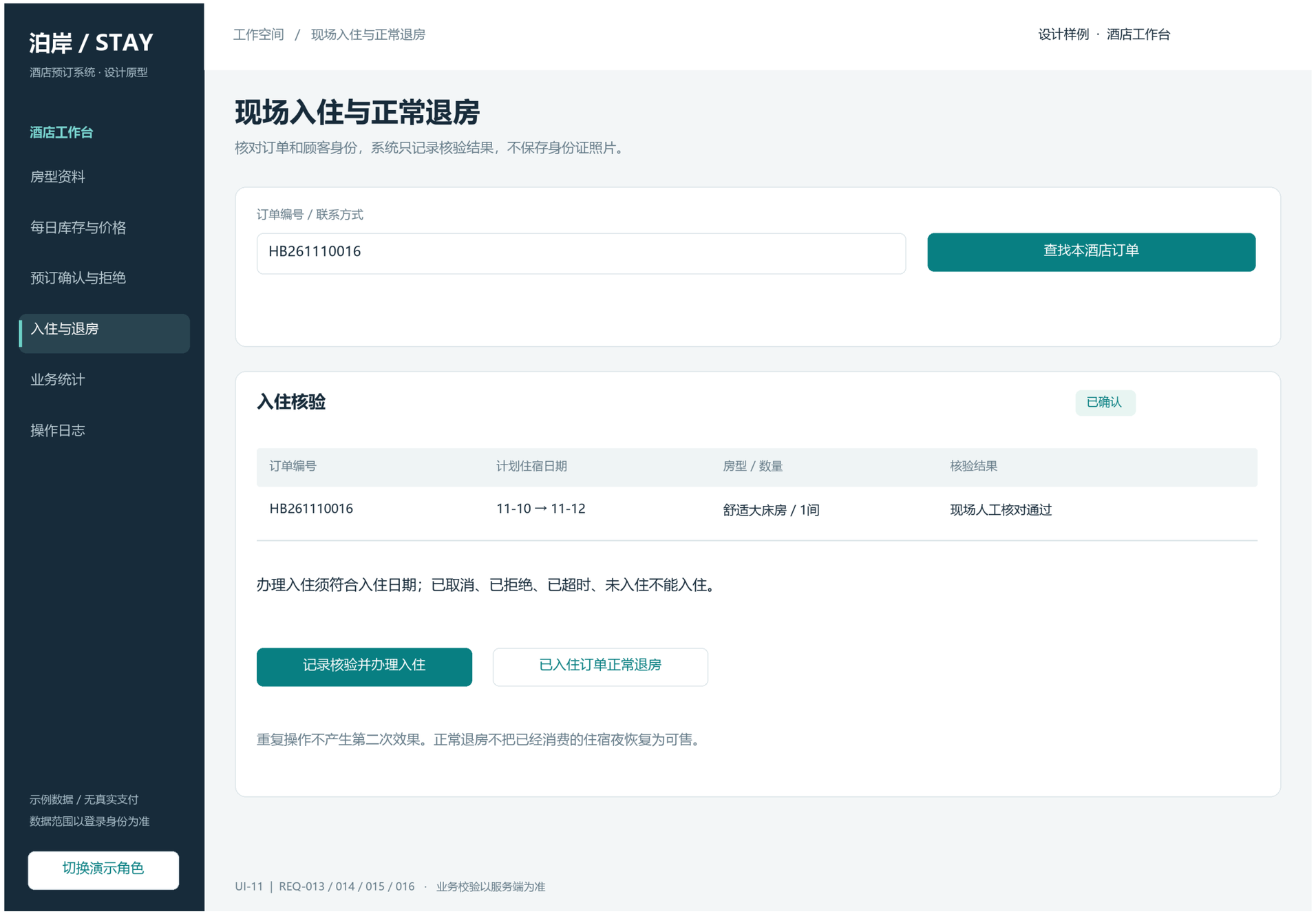
UI-10 预订确认与拒绝

酒店工作台，对应REQ-009 / 010 / 016。按本酒店、当前状态与版本处理；其他酒店订单不可访问。



UI-11 现场入住与正常退房

酒店工作台，对应REQ-013 / 014 / 015 / 016。核对订单和顾客身份，系统只记录核验结果，不保存身份证照片。



UI-12 酒店审核、启停与异常处置

平台管理台，对应REQ-002 / 017 / 019。停业阻止新预订；已有订单分类处理，不能抹去历史记录。



UI-13 工作人员账号管理

平台管理台，对应REQ-001 / 020 / 023。每名工作人员绑定唯一酒店；停用账号不删除历史操作记录。



UI-14 预订数量与间夜统计

酒店工作台，对应REQ-018 / 001。工作人员限本酒店，管理员可切换全平台；不统计收入或入住率。



UI-15 操作日志与追踪

酒店工作台，对应REQ-019 / 023 / 024。日志只读、保留180天；管理员查全平台，酒店人员只查本酒店。



UI-16 移动端订单详情

顾客端，对应REQ-025。手机关键任务适配

